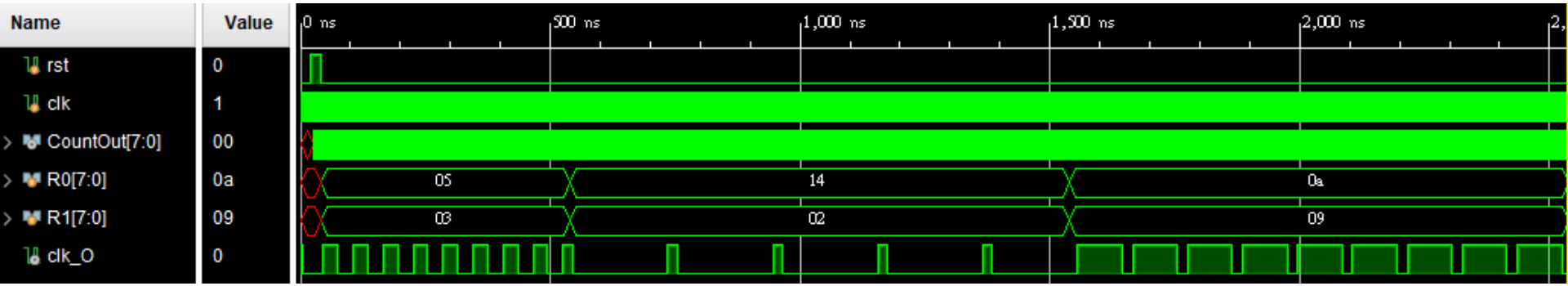
Clock Divider:

```
initial begin
    rst= 1'b0;
    clk= 1'b0;

    //#30   rst=1'b1;
    #`halfT
    #`halfT
    #`halfT
    #`halfT  rst=1'b1;
    #`halfT
    #`halfT
    #`halfT
    #`halfT  rst=1'b0;
             R0=8'd5;
             R1=8'd3;

    # 500    R0=8'd20;
             R1=8'd2;

    # 1000    R0=8'd10;
              R1=8'd9;

    # 1000   $stop;

end
```

```
`timescale 1ns / 1ps
module ClkDiv(clk, rst, CountOut, clk_O, R0, R1);
input clk, rst;
output reg [7:0] CountOut;
output reg clk_O;
input [7:0] R0, R1;

always@(posedge clk) begin
    if( rst==1'b1 || CountOut>=R0 ) begin
        CountOut <= 8'h00;
    end
    else begin
        CountOut <= CountOut+1'b1;
    end
end

always@(posedge clk) begin
    if( rst==1'b1 ) begin
        clk_O <= 1'b0;
    end
    else begin
        if( CountOut < R1 ) begin
            clk_O <= 1'b1;
        end
        else begin
            clk_O <= 1'b0;
        end
    end
end

endmodule
```


Name
Value
rst
0
clk
1
CountOut[7:0]
00
R0[7:0]
0a
R1[7:0]
09
clk_O
0
0 ns
500 ns
1,000 ns
1,500 ns
2,000 ns
05
14
0a
03
02
09

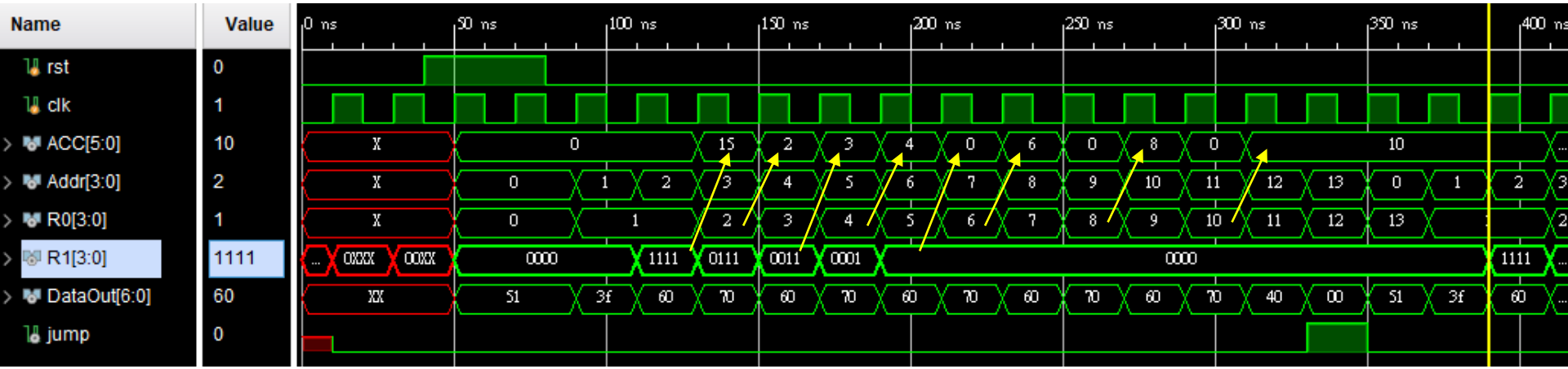
Add more registers in CPU: (in addition to ACC)

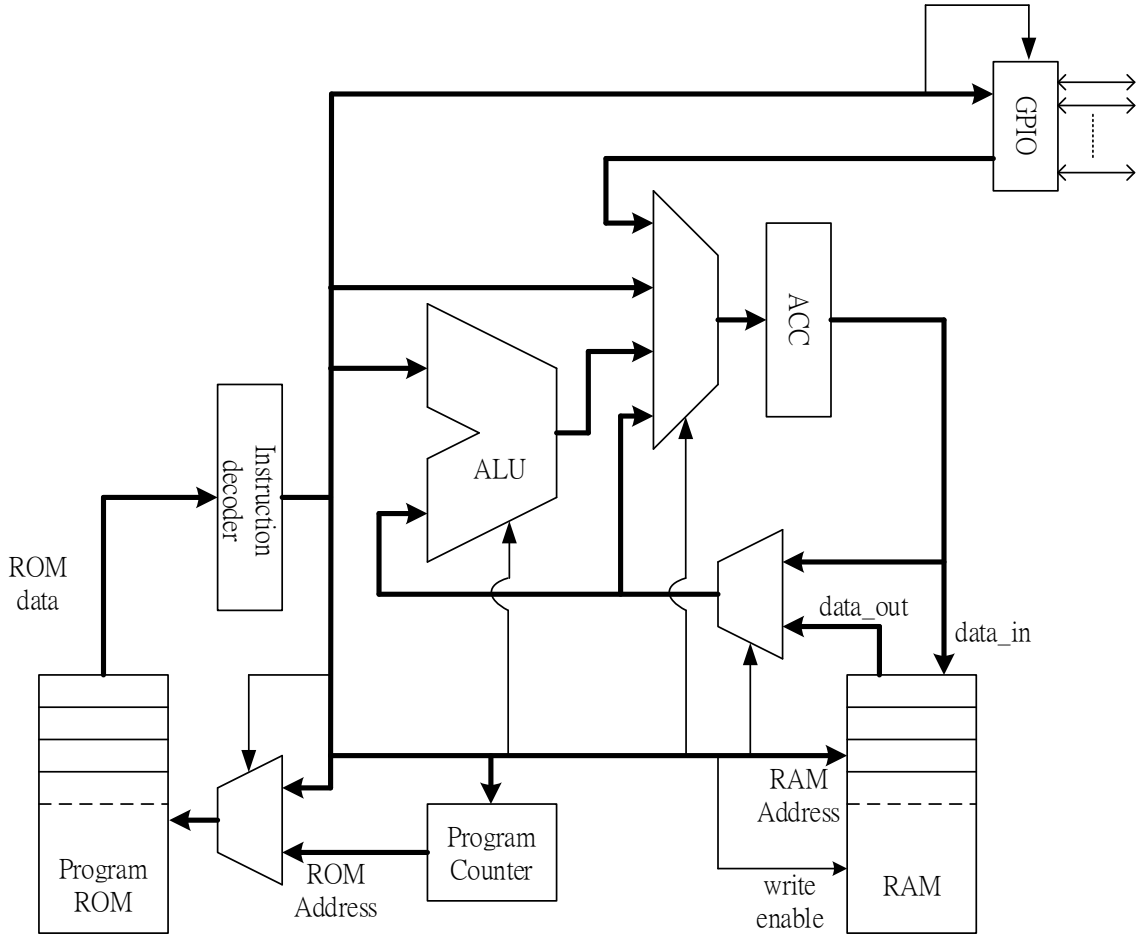
```
`timescale 1ns / 1ps
module CPU_1(clk, rst, ACC, Addr, DataOut, jump, JpAddr, R0, R1);
input clk, rst;
output reg [5:0] ACC;
output reg [3:0] Addr;
output reg [6:0] DataOut;
output reg jump;
output reg [3:0] JpAddr;
output reg [3:0] R0, R1;
```

```
// Other Registers
always@(posedge clk) begin
    if(rst == 1'b1) begin
        R0 <= 4'b0000;
        R1 <= 4'b0000;
    end
    else begin
        case ( DataOut[6:4] )
            3'b101: R0 <= DataOut[3:0];    // Set R0
            3'b011: R1 <= DataOut[3:0];    // Set R1
            default: begin
                R0 <= R0+1'b1;
                R1 <= R1 >>1;
            end
        endcase
    end
end
```

```
always@(posedge clk) begin
    if(rst == 1'b1) begin
        ACC <= 6'b00_0000;
    end
    else begin
        case ( DataOut[6:4] )
            3'b001: ACC <= {2'b00, DataOut[3:0]}; // Set
            3'b010: ACC <= ACC + DataOut[3:0];    // Add
            3'b111: ACC <= R0;                    // Read R0
            3'b110: ACC <= R1;                    // Read R1
            default: ACC <= ACC;
        endcase
    end
end
```

```
always @(*)begin
    case ( Addr )
        4'b0000: DataOut = 7'b101_0001; // Set R0=4'b0001
        4'b0001: DataOut = 7'b011_1111; // Set R1=4'b1111
        4'b0010: DataOut = 7'b110_0000; // Read R1 to ACC
        4'b0011: DataOut = 7'b111_0000; // Read R0 to ACC
        4'b0100: DataOut = 7'b110_0000; // Read R1 to ACC
        4'b0101: DataOut = 7'b111_0000; // Read R0 to ACC
        4'b0110: DataOut = 7'b110_0000; // Read R1 to ACC
        4'b0111: DataOut = 7'b111_0000; // Read R0 to ACC
        4'b1000: DataOut = 7'b110_0000; // Read R1 to ACC
        4'b1001: DataOut = 7'b111_0000; // Read R0 to ACC
        4'b1010: DataOut = 7'b110_0000; // Read R1 to ACC
        4'b1011: DataOut = 7'b111_0000; // Read R0 to ACC
        4'b1100: DataOut = 7'b100_0000; // Jump to Addr=4'b0000
        default: DataOut = 7'b000_0000;
    endcase
end
```

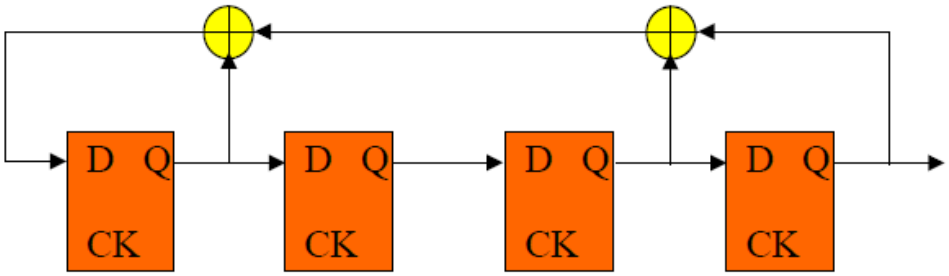

Name
Value
rst
clk
ACC[5:0]
Addr[3:0]
R0[3:0]
R1[3:0]
DataOut[6:0]
jump
0 ns
50 ns
100 ns
150 ns
200 ns
250 ns
300 ns
350 ns
400 ns


GPIO
ACC
ALU
Instruction decoder
ROM data
data_out
data_in
Program ROM
ROM Address
Program Counter
RAM Address
write enable
RAM

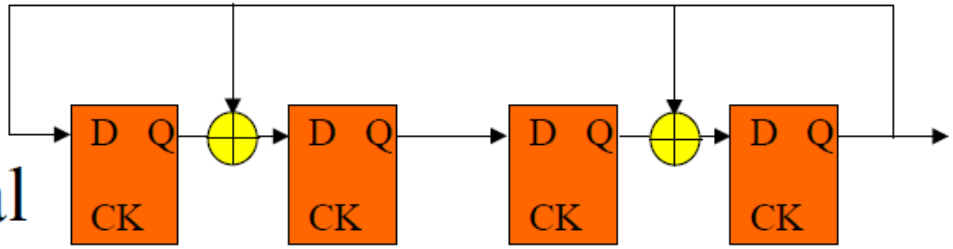
Linear Feedback Shift Registers (LFSRs)

- Efficient design for Test Pattern Generators & Output Response Analyzers (also used in CRC)
  - FFs plus a few XOR gates
  - better than counter
    - fewer gates
    - higher clock frequency
- Two types of LFSRs
  - External Feedback
  - Internal Feedback
    - higher clock frequency
- Characteristic polynomial
  - defined by XOR positions
  - $P(x) = x^4 + x^3 + x + 1$ in both examples

External Feedback LFSR



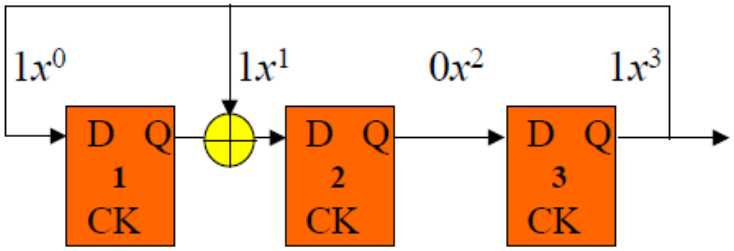
Internal Feedback LFSR



LFSRs (cont)

Characteristic polynomial of LFSR

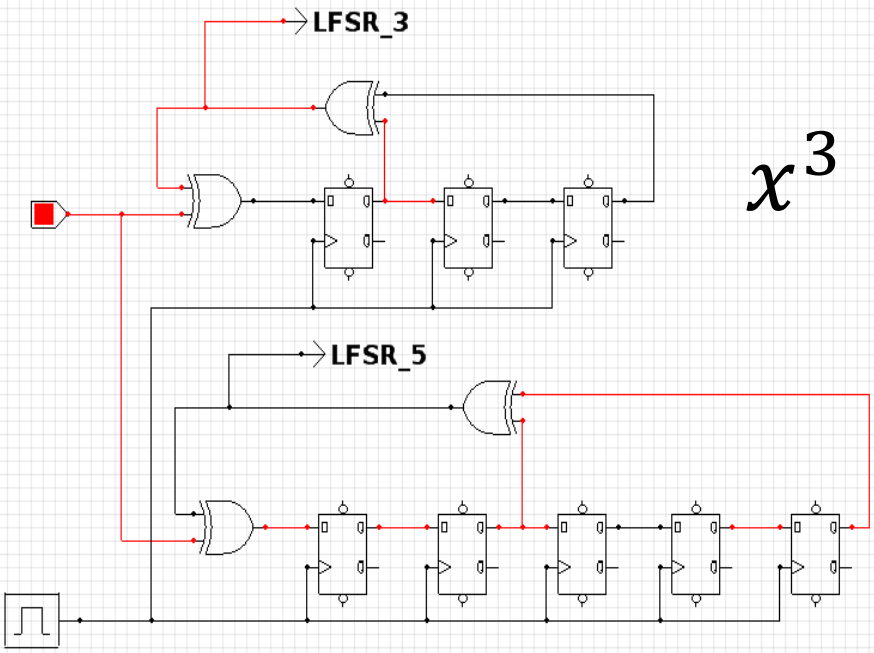
- n = # of FFs = degree of polynomial
- XOR feedback connection to FF $i \Leftrightarrow$ coefficient of x^i
  - coefficient = 0 if no connection
  - coefficient = 1 if connection
  - coefficients always included in characteristic polynomial:
    - x^n (degree of polynomial & primary feedback)
    - $x^0 = 1$ (principle input to shift register)
- Note: state of the LFSR $\Leftrightarrow$ polynomial of degree n-1
- Example: $P(x) = x^3 + x + 1$



LSFRs (cont)

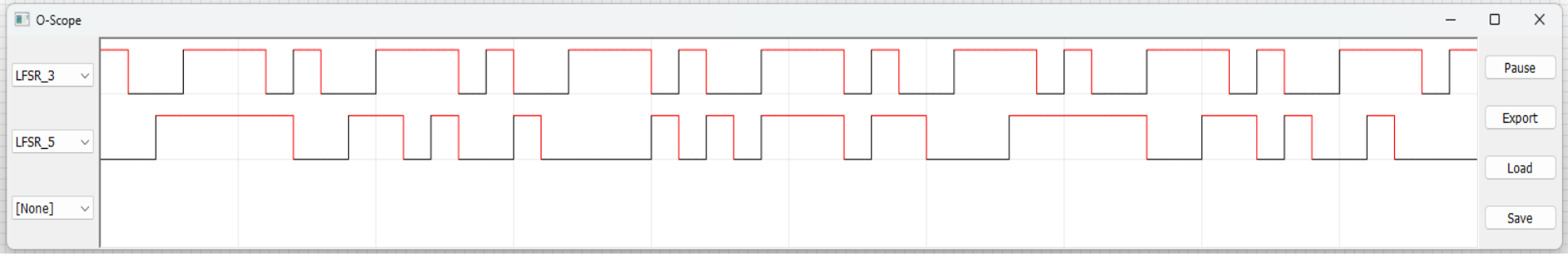
- An LFSR generates periodic sequence
  - must start in a non-zero state,
- The maximum-length of an LFSR sequence is 2^n -1
  - does not generate all 0s pattern (gets stuck in that state)
- The characteristic polynomial of an LFSR generating a maximum-length sequence is a ***primitive polynomial***
- A maximum-length sequence is ***pseudo-random***:
  - number of 1s = number of 0s + 1
  - same number of runs of consectuive 0s and 1s
  - 1/2 of the runs have length 1
  - 1/4 of the runs have length 2
  - … (as long as fractions result in integral numbers of runs)

Generate pseudo random sequences; length $=2^n - 1$



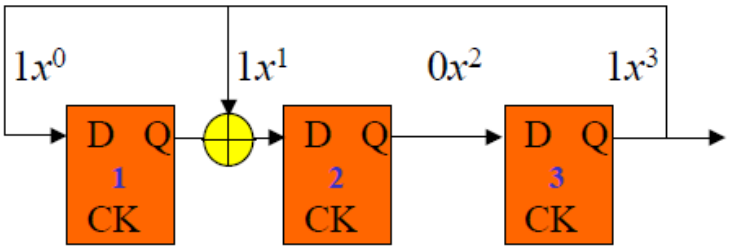
$$x^3 + x + 1, \text{ n=3}$$

$$x^5 + x^2 + 1, \text{ n=5}$$



LFSRs (cont)

- Example: Characteristic polynomial is $P(x) = x^3 + x + 1$
- Beginning at all 1s state
  - 7 clock cycles to repeat
  - maximal length = 2^n-1
  - polynomial is primitive
- Properties:
  - four 1s and three 0s
  - 4 runs:
    - 2 runs of length 1 (one 0 & one 1)
    - 1 run of length 2 (0s)
    - 1 run of length 3 (1s)



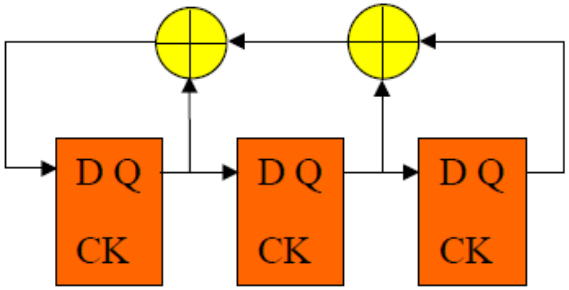
1	1	1	1
1	0	1	2
1	0	0	3
0	1	0	4
0	0	1	5
1	1	0	6
0	1	1	7
1	1	1	

- Note: external & internal LFSRs with same primitive polynomial do not generate same sequence (only same length)

LFSRs (cont)

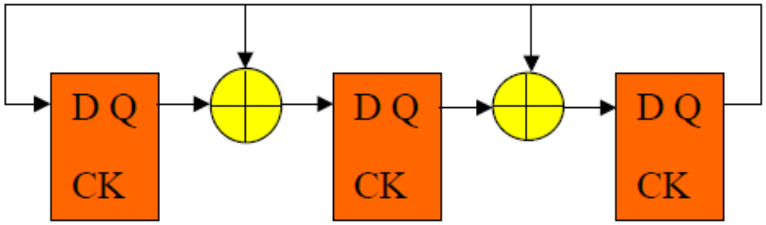
- Non-primitive polynomials produce sequences < 2^n-1
  - Typically primitive polys desired for TPGs & ORAs
- Example of non-primitive polynomial
  - $P(x) = x^3 + x^2 + x + 1$

External Feedback LFSR



0 0 0	1 1 1	1 0 0	0 1 0
0 0 0	1 1 1	1 1 0	1 0 1
		0 1 1	0 1 0
		0 0 1	
		1 0 0	

Internal Feedback LFSR

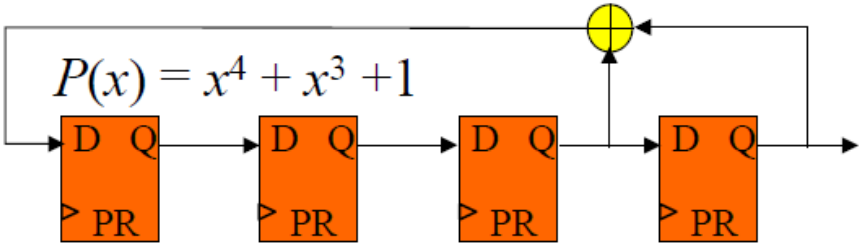


0 0 0	1 1 1	1 1 0	1 0 1
0 0 0	1 0 0	0 1 1	1 0 1
	0 1 0	1 1 0	
	0 0 1		
	1 1 1		

Linear Feedback Shift Register & Primitive Polynomial

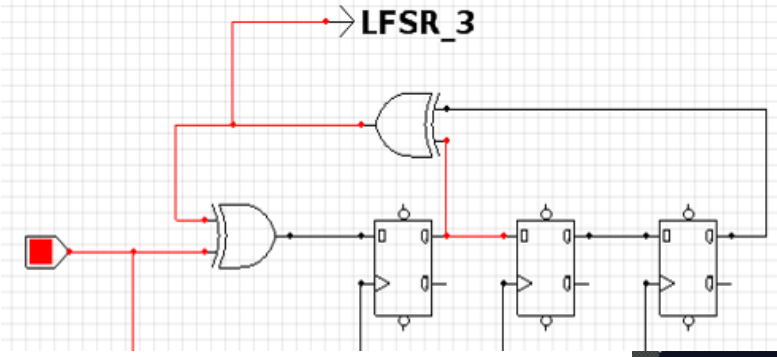
Degree (n)	Polynomial
2,3,4,6,7,15,22	$x^n + x + 1$
5,11,21,29	$x^n + x^2 + 1$
8,19	$x^n + x^6 + x^5 + x + 1$
9	$x^n + x^4 + 1$
10,17,20,25,28	$x^n + x^3 + 1$
12	$x^n + x^7 + x^4 + x^3 + 1$
13,24	$x^n + x^4 + x^3 + x + 1$
14	$x^n + x^{12} + x^{11} + x + 1$
16	$x^n + x^5 + x^3 + x^2 + 1$
18	$x^n + x^7 + 1$
23	$x^n + x^5 + 1$
26,27	$x^n + x^8 + x^7 + x + 1$
30	$x^n + x^{16} + x^{15} + x + 1$

HDL descriptions




```
-- VHDL model
entity LFSR is
port (CLK,PR: in bit;
        Q:  buffer bit_vector(4 downto 1));
end entity LFSR;
architecture RTL of LFSR is
begin
process (CLK)
begin
if (CLK'event and CLK = '1') then
    if (PR = '1') then
        Q <= "1111";
    else
        for K in 2 to 4 loop
            Q(K) <= Q(K-1);
        end loop;
    end if;
    Q(1) <= Q(4) xor Q(3);
end if;
end process;
end architecture RTL;
```

```
// Verilog model
module LFSR (CLK, PR, Q);
input CLK, PR;
output [4:1] Q;
/* end of "entity" portion and
    beginning of "architecture" portion */
reg Q;
integer K;
always @ (posedge CLK) begin
    if (PR == 1)
        Q = 4'b1111;
    else begin
        for (K=2 ; K < 5 ; K=K+1) begin
            Q[K] = Q[K-1];
        end
        Q[1] = Q[4] ^ Q[3];
    end
end
endmodule
```



$$x^3 + x + 1, \text{n=3}$$

```
`timescale 1ns / 1ps
module LFSR(clk, rst, In, Out, ShiftReg3);
input clk, rst, In;
output reg [2:0] ShiftReg3;
output Out;

always@(posedge clk) begin
    if(rst == 1'b1) begin
        ShiftReg3 <= 3'b000;
    end
    else begin
        ShiftReg3 <= ShiftReg3<<1;
        ShiftReg3[0] <= Out^In;
    end
end

assign Out = ShiftReg3[0]^ShiftReg3[2];

endmodule
```

```
module LFSR_2to30(clk, rst, In, Out, ShiftReg);
    parameter Length=5'd4;
    integer idx;

    input clk, rst, In;
    output reg [Length-1:0] ShiftReg;
    output reg Out;

    always@(posedge clk or posedge rst) begin
        if(rst == 1'b1) begin
            for (idx = 0; idx < Length; idx = idx + 1) begin
                ShiftReg[idx] <= 1'b0;
            end
        end
        else begin
            ShiftReg <= ShiftReg>>1;
            ShiftReg[Length-1] <= Out^In;
        end
    end

    always@(*) begin
        case(Length)
    end
endmodule
```

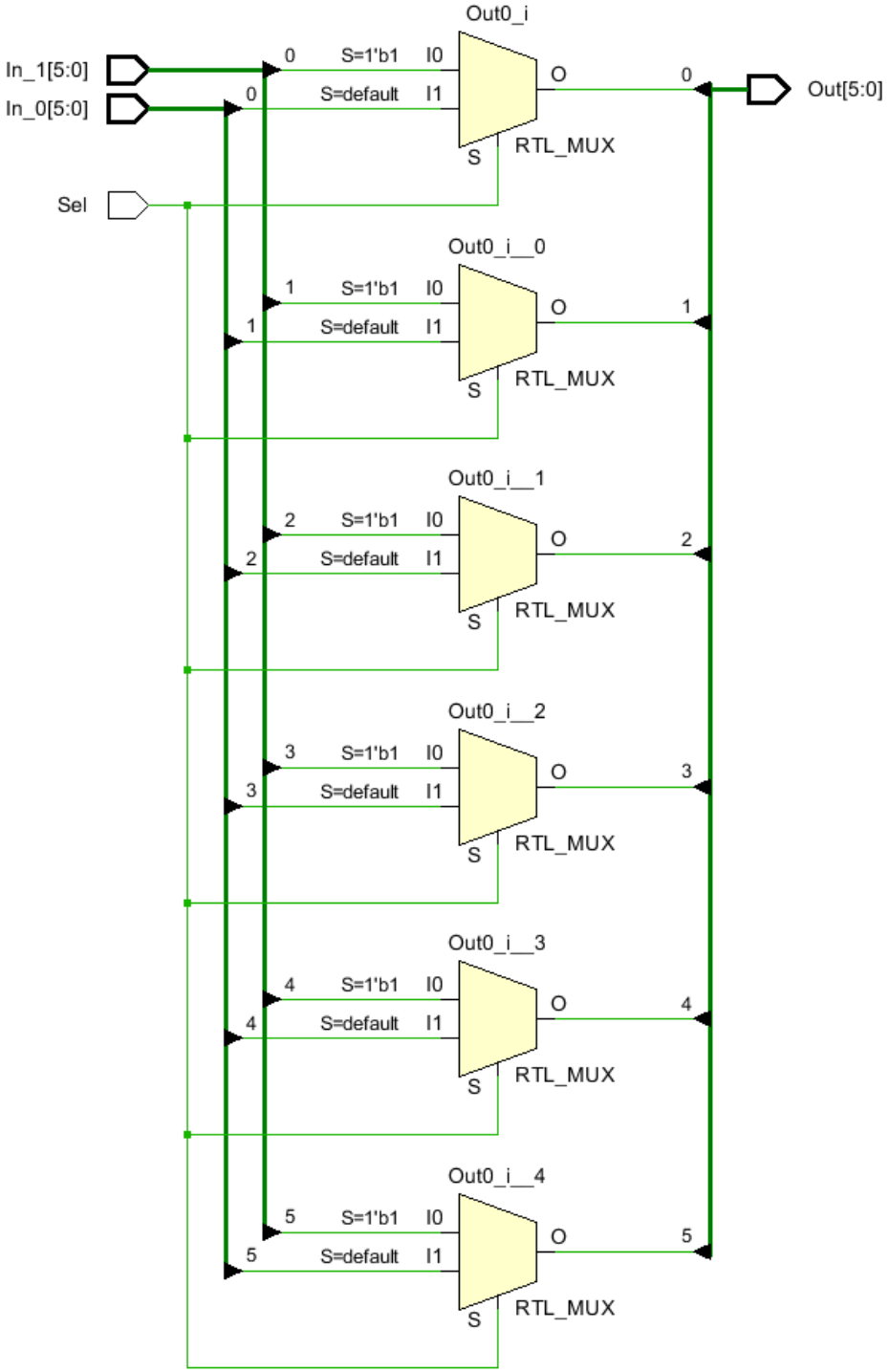
```
always@(*) begin
    case(Length)
    2,3,4,6,7,15,22:  Out = ShiftReg[1]^ShiftReg[0];
    5,11,21,29:       Out = ShiftReg[2]^ShiftReg[0];
    8,19:             Out = ShiftReg[6]^ShiftReg[5]^ShiftReg[1]^ShiftReg[0];
    9:                Out = ShiftReg[4]^ShiftReg[0];
    10,17,20,25,28:   Out = ShiftReg[3]^ShiftReg[0];
    12:               Out = ShiftReg[7]^ShiftReg[4]^ShiftReg[3]^ShiftReg[0];
    13,24:            Out = ShiftReg[4]^ShiftReg[3]^ShiftReg[1]^ShiftReg[0];
    14:               Out = ShiftReg[12]^ShiftReg[11]^ShiftReg[1]^ShiftReg[0];
    16:               Out = ShiftReg[5]^ShiftReg[3]^ShiftReg[2]^ShiftReg[0];
    18:               Out = ShiftReg[7]^ShiftReg[0];
    23:               Out = ShiftReg[5]^ShiftReg[0];
    26,27:            Out = ShiftReg[8]^ShiftReg[7]^ShiftReg[1]^ShiftReg[0];
    30:               Out = ShiftReg[16]^ShiftReg[15]^ShiftReg[1]^ShiftReg[0];
    default:          Out = ShiftReg[1]^ShiftReg[0];
    endcase
end
```

For loop example 1: mux

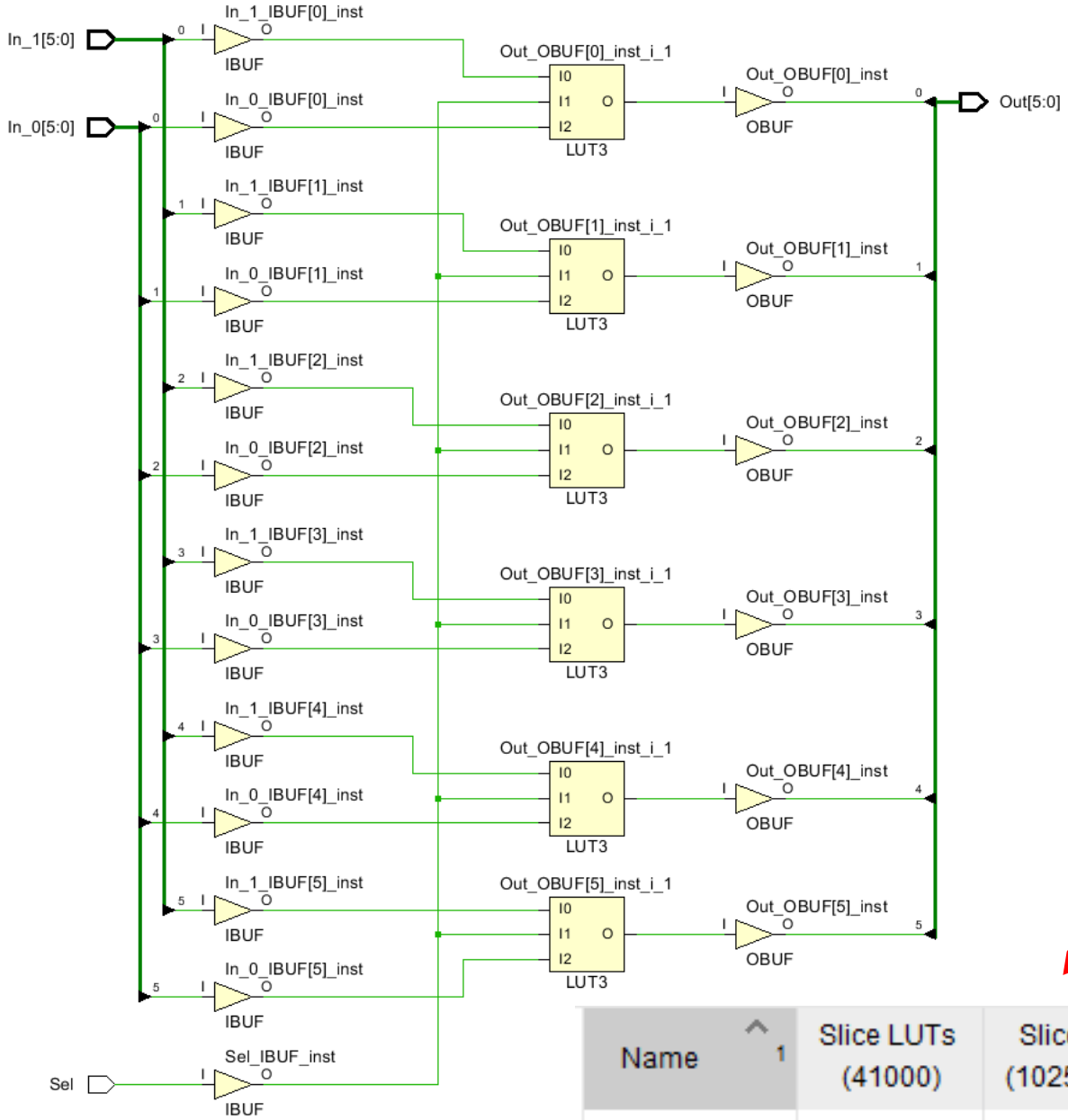
```
module ForMux(In_1, In_0, Sel, Out);
parameter BusWidth=5;

input [BusWidth:0]In_1;
input [BusWidth:0]In_0;
input Sel;
output [BusWidth:0]Out;
reg    [BusWidth:0]Out;

integer idx;

always begin
    for (idx = 0; idx <= BusWidth; idx = idx + 1) begin
        Out[idx]=Sel ? In_1[idx] : In_0[idx];
    end
end

//assign Out=Sel ? In_1 : In_0;

endmodule
```

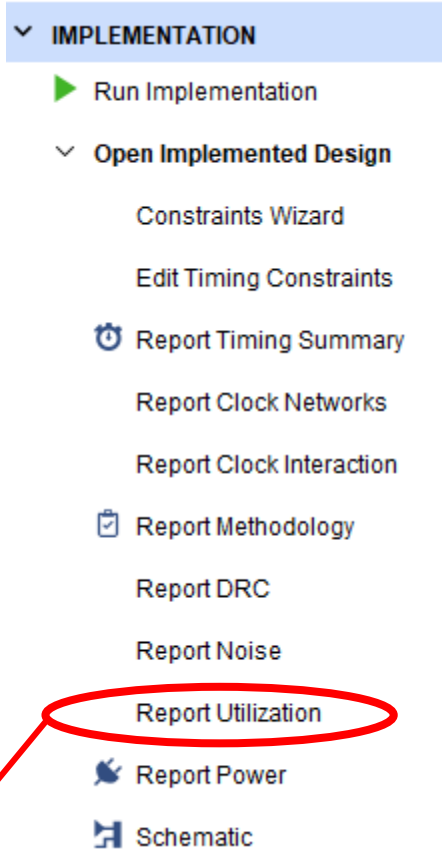
Set the parameter

Elaborate the mux:



Implement the mux: use only LUT (combinational logic)



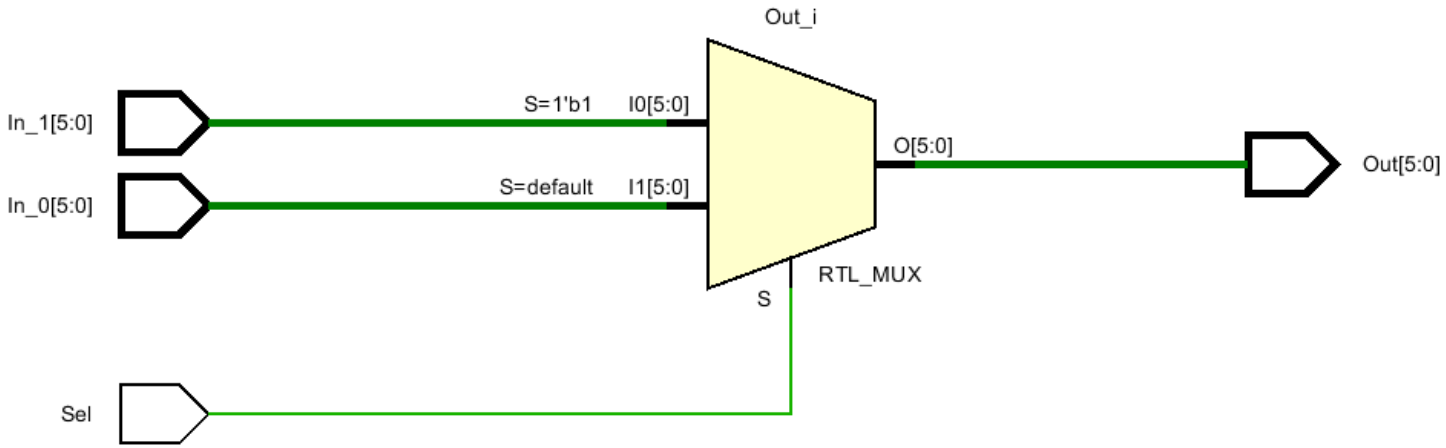


Name	Slice LUTs (41000)	Slice (10250)	LUT as Logic (41000)	Bonded IPADs (26)
ForMux	3	3	3	19

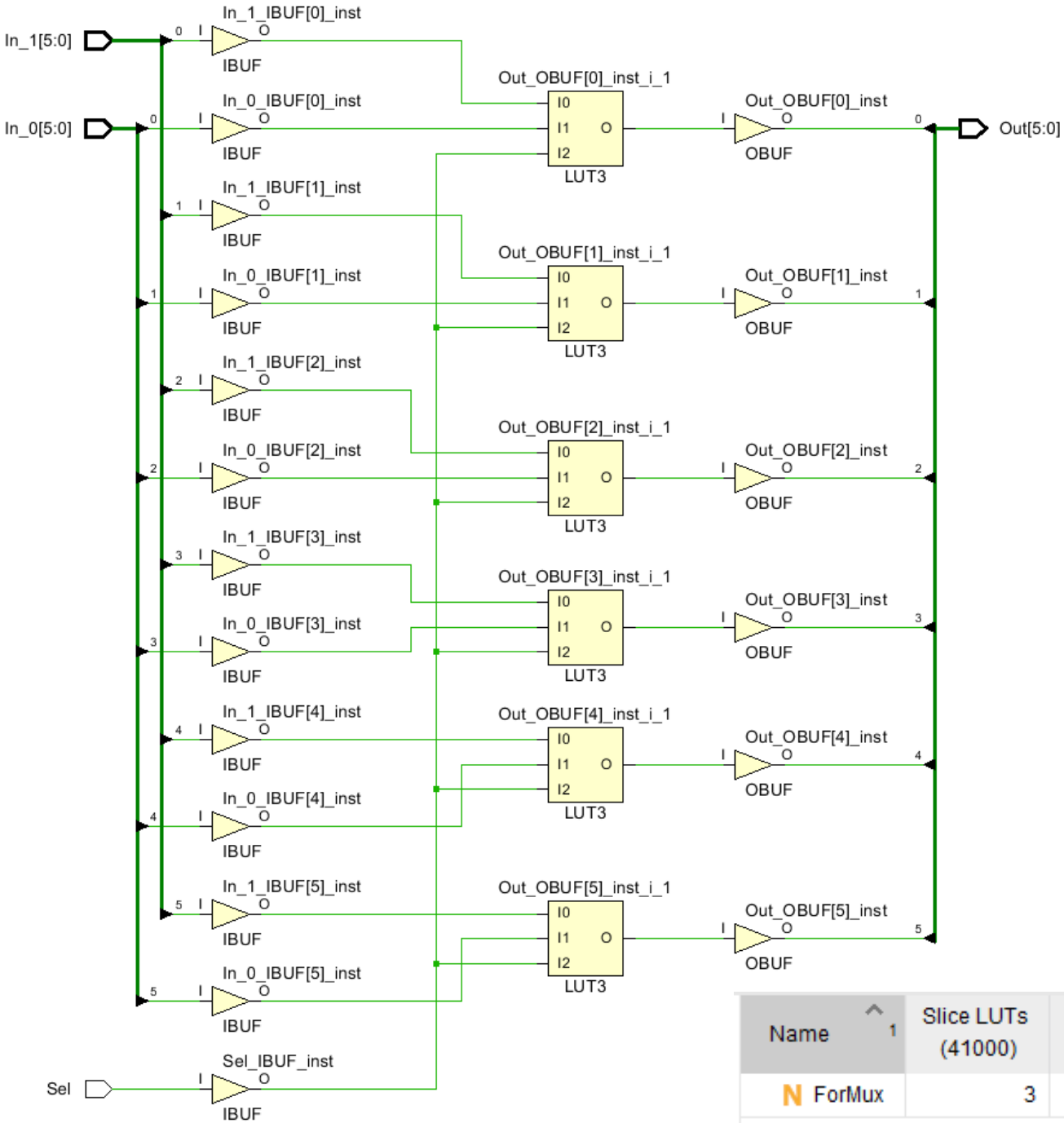
A normal mux:

```
module ForMux(In_1, In_0, Sel, Out);
parameter BusWidth=5;

input [BusWidth:0]In_1;
input [BusWidth:0]In_0;
input Sel;
output [BusWidth:0]Out;
//reg    [BusWidth:0]Out;

//integer idx;

//always begin
//    for (idx = 0; idx <= BusWidth; idx = idx + 1) begin
//      Out[idx]=Sel ? In_1[idx] : In_0[idx];
//  end
//end

assign Out=Sel ? In_1 : In_0;

endmodule
```

Elaborate:



implementation:

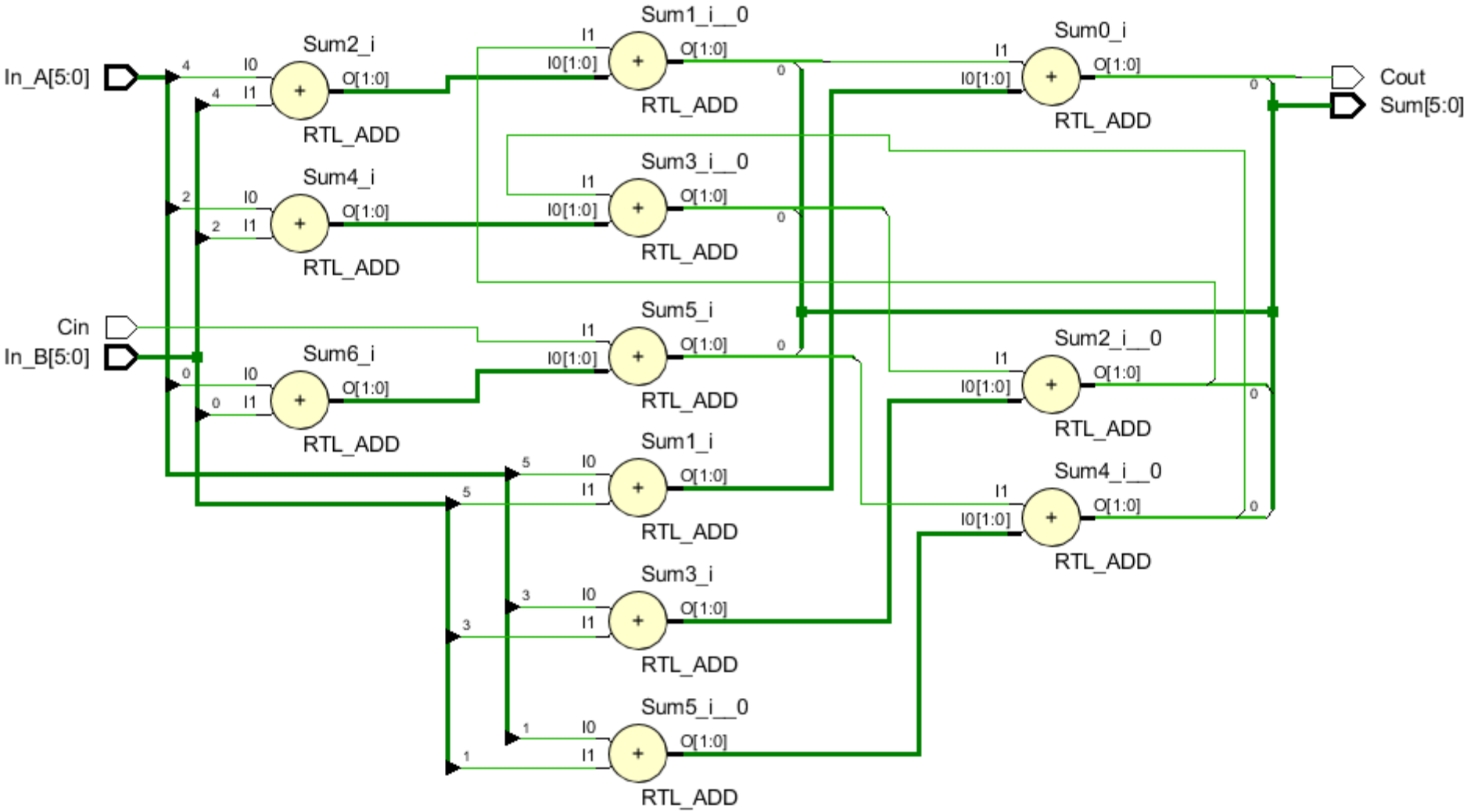


Name	Slice LUTs (41000)	Slice (10250)	LUT as Logic (41000)	Bonded IPADs (26)
ForMux	3	3	3	19

For loop example 2: adder

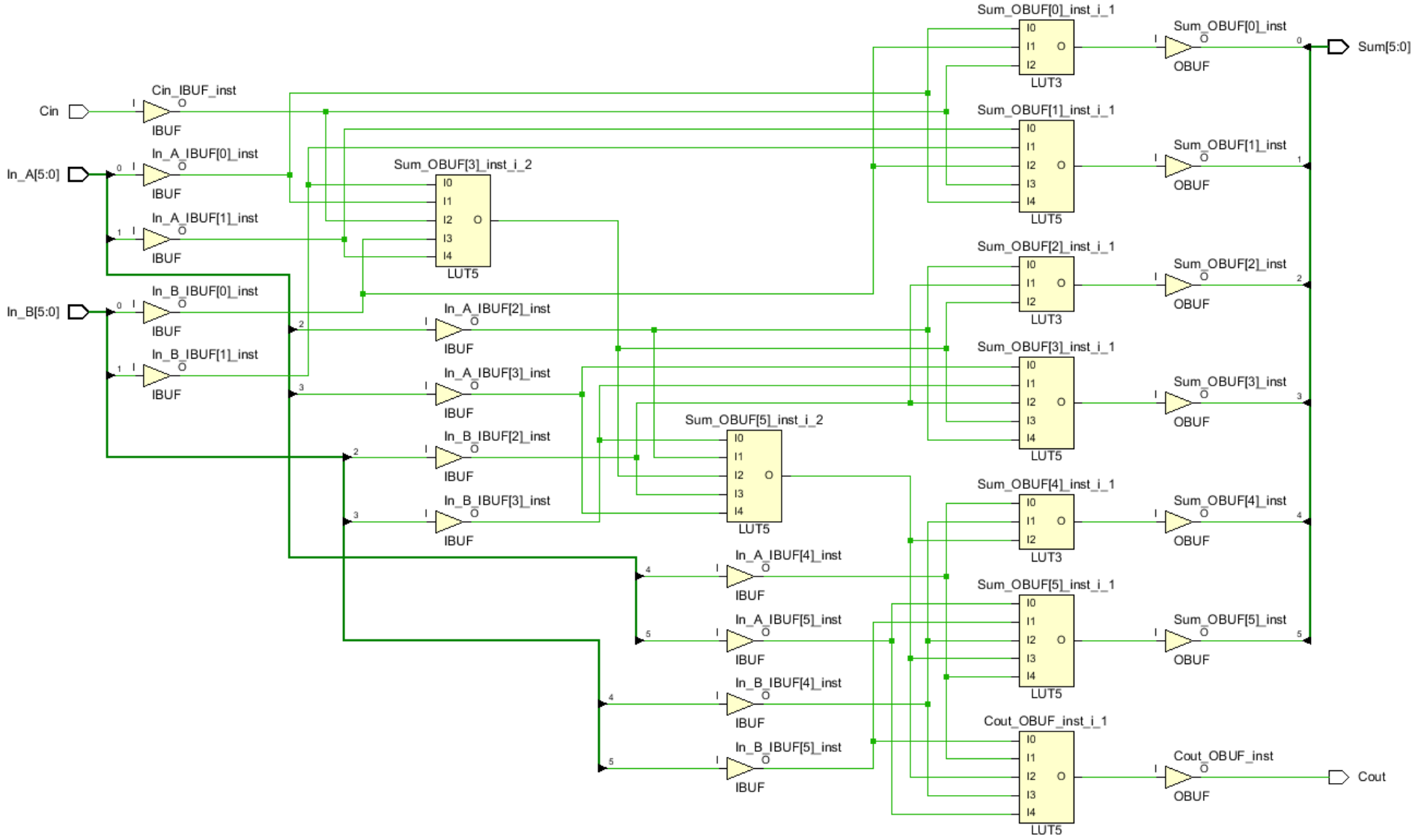
```
module ForAdder(In_A, In_B, Cin, Sum, Cout);
parameter BusWidth=5;

input [BusWidth:0]In_A;
input [BusWidth:0]In_B;
input Cin;
output [BusWidth:0]Sum;
reg    [BusWidth:0]Sum;
output Cout;

reg    [BusWidth:0] C_i;

integer idx;

assign Cout=C_i[BusWidth];

always begin
    {C_i[0], Sum[0]} = In_A[0] + In_B[0] + Cin;
end


always begin
    for (idx = 1; idx <= BusWidth; idx = idx + 1) begin
        {C_i[idx], Sum[idx]} = In_A[idx] + In_B[idx] + C_i[idx-1];
    end
end

endmodule
```

Elaborate the adder:



Implement the adder:

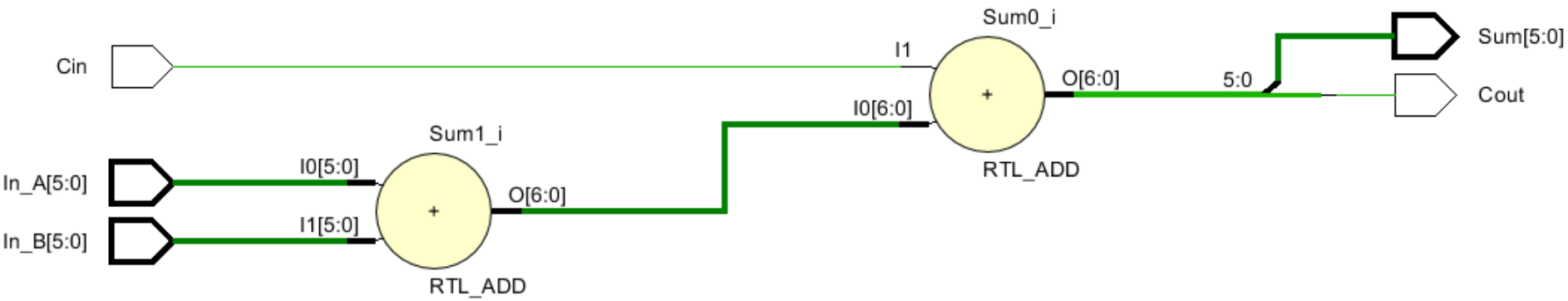
Name	Slice LUTs (41000)	Slice (10250)	LUT as Logic (41000)	Bonded IPADs (26)
N ForAdder	6	2	6	20



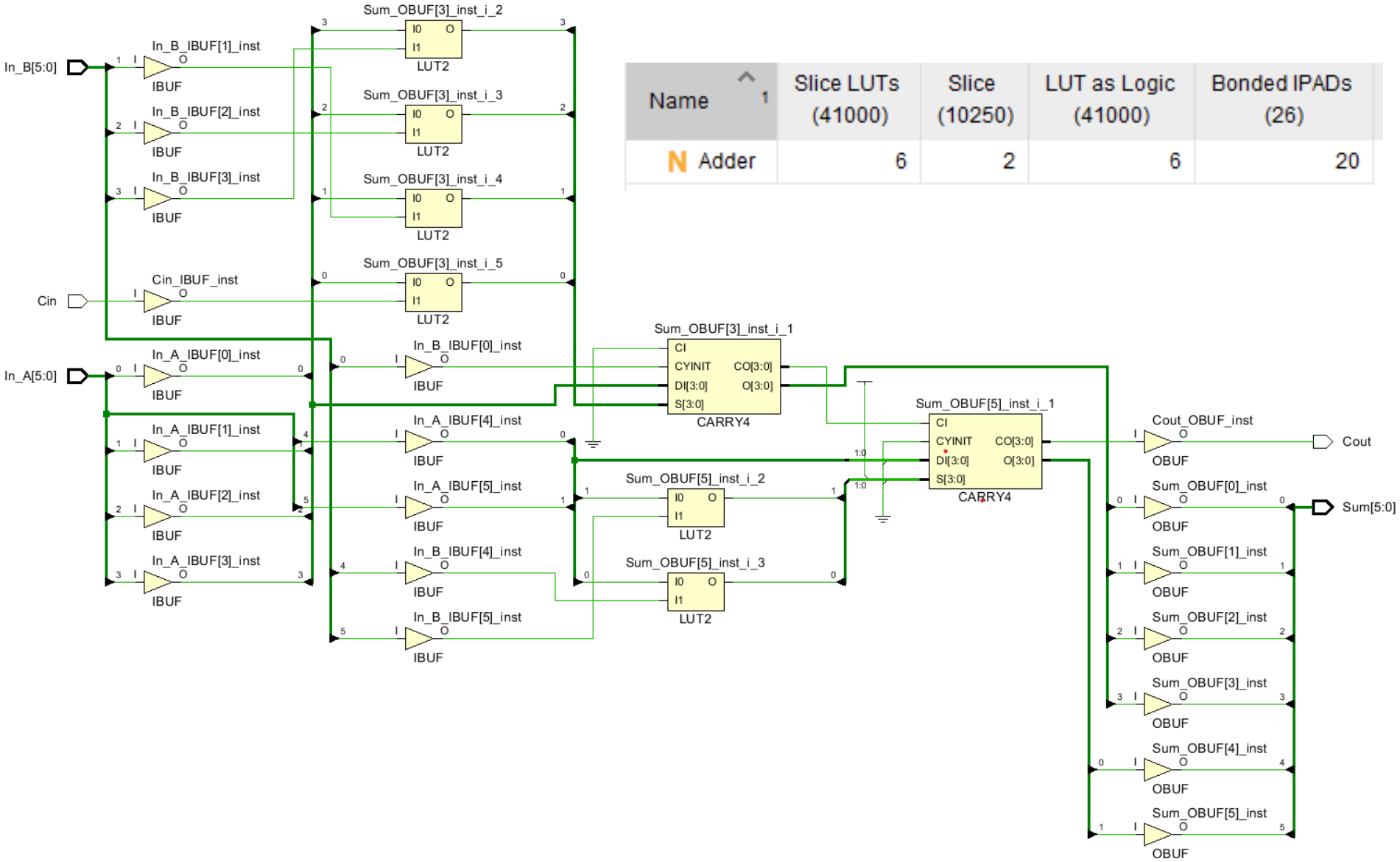
An adder:

```
module Adder(In_A, In_B, Cin, Sum, Cout);
parameter BusWidth=5;

input [BusWidth:0]In_A;
input [BusWidth:0]In_B;
input Cin;
output [BusWidth:0]Sum;

output Cout;

assign {Cout, Sum}=In_A + In_B + Cin;

endmodule
```

Elaborate:



Implementation:



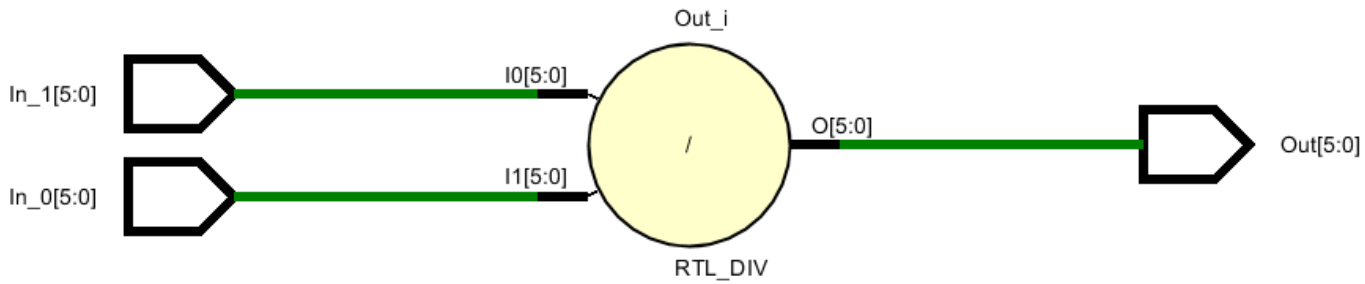
Name	Slice LUTs (41000)	Slice (10250)	LUT as Logic (41000)	Bonded IPADs (26)
N Adder	6	2	6	20

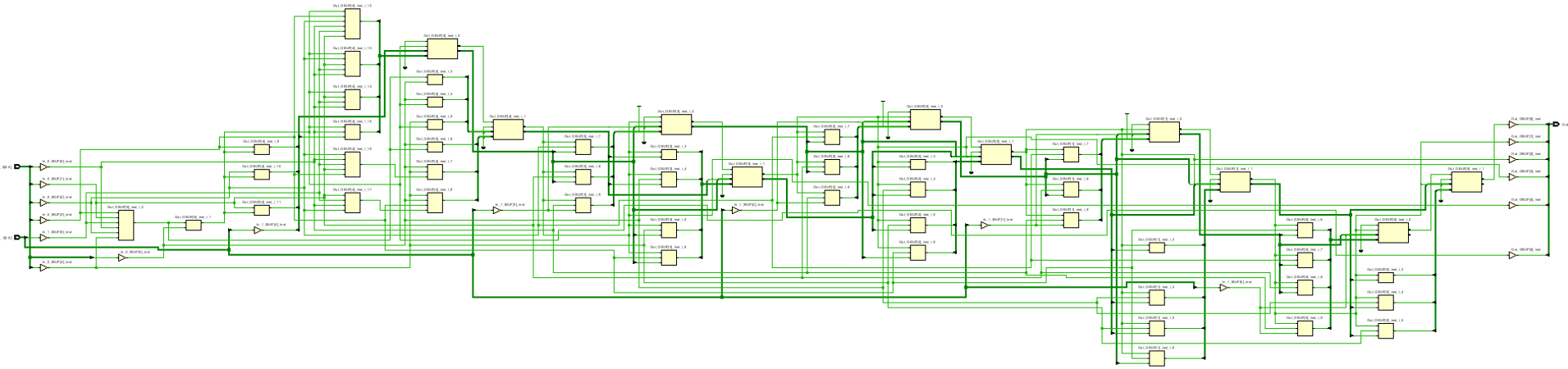
Divider:

```
module Divider(In_1, In_0, Out);
parameter BusWidth=5;

input [BusWidth:0]In_1;
input [BusWidth:0]In_0;
output [BusWidth:0]Out;

assign Out= In_1 / In_0;

endmodule
```

Name	Slice LUTs (41000)	Slice (10250)	LUT as Logic (41000)	Bonded IPADs (26)
Divider	39	11	39	18

Elaborate & Implementation:





Behavioral an data-flow

Synthesizable

always @(*)（combinational logic）
always @(posedge clk)（sequential logic）
always @(negedge clk)
assign（ combinational logic ）

generate（used with fixed “parameter” ）
function（only for combinational functions）

Unsynthesizable

initial
#（delay times）
Wait
fork / join

Modules & Hierarchy

Synthesizable

module
input / output / inout
assign
always
instance
generate

Unsynthesizable

task（only for simulation）
deassign
force / release

Task & Function

```
`timescale 1ns / 1ps

module TB_Task;

reg [7:0] A, B;
reg [8:0] Sum;

  task sumAB;
    input [7:0] x, y;
    output [7:0] sum;
    begin
      sum = x + y;
      $display("Sum of %d
    end
  endtask

initial begin

    A = 7'd10;
    B = 7'd5;
    sumAB(A, B, Sum);
#100

    A = 7'd15;
    B = 7'd4;
    sumAB(A, B, Sum);
#100
    $stop;
end
endmodule
```

```
`timescale 1ns / 1ps

module TB_Func;

reg [7:0] A, B;
reg [8:0] Sum;

  function [8:0] sumAB;
    input [7:0] x, y;
    begin
      sumAB = x + y;
      $display("Sum of %d and %d is %d", x, y, sumAB);
    end
  endfunction

initial begin

    A = 7'd10;
    B = 7'd5;
    Sum=sumAB(A, B);
#100

    A = 7'd15;
    B = 7'd4;
    Sum=sumAB(A, B);
#100
    $stop;
end

endmodule
```

比較項目	module	task / function
對應硬體	✅ 實體硬體模組	❌ 僅程式邏輯片段
可被綜合	✅ 是主要合成單位	⚠️ 有限制才能合成
呼叫方式	例化（像插電路板）	像呼叫函式（在 always block 內）
參數類型	輸入、輸出、雙向	task: in/out/inout function: input only, return value
重用程度	高（IP core）	高（內部重複邏輯）

項目	`module`	`function`	`task`
🔗 語意	獨立硬體模組	可重用的組合邏輯	可重用的邏輯步驟（含控制流程）
🛠️ 可合成	✅ 是	✅ 是（符合條件下）	⚠️ 僅在完全組合時可合成
⚙️ 呼叫方式	例化	表達式中呼叫	在 `always` 區塊中呼叫
🔁 輸出數量	多個輸出可用	只能 return 一個值	可同時輸出多個值
👨‍🔧 用途	建構硬體子系統	數學邏輯、轉換、比較等	複雜邏輯流程，或行為模擬
🧪 常見用在	RTL 設計主模組	RTL 組合邏輯、解碼器	Testbench、控制邏輯